EECE 2323 Digital Logic Design Lab

Lab Report 6: Adding Instruction Memory and Program Counter to Your Computer

Name: Arefin Mahmood

Date: 06/10/2026

Background & Purpose

In this lab, I added instruction memory and a Program Counter to the computer from Lab 5. In Lab 5, the instruction fields were still manually controlled through VIO. In Lab 6, the computer reads 16-bit machine instructions from instruction memory instead. The Program Counter gives the current instruction address, instruction memory outputs the instruction stored at that address, the mapper separates the instruction into fields, and the instruction decoder generates the control signals for the datapath. This lab is important because a real processor does not manually receive rs1, rs2, rd, opcode, and immediate values from outside inputs. It fetches stored instructions from memory and executes them in order.

Design Analysis

The main change from Lab 5 was adding instruction fetch hardware. I added an 8-bit Program Counter and connected its output to instruction memory. The PC resets to 0 when BTN0 is pressed and increments by 1 on each debounced BTN1 clock press. Since Lab 6 only uses straight-line code, the PC does not branch or jump. It simply moves through the instruction memory one address at a time.

Instruction memory was generated as a 256-entry, 16-bit ROM using the testing_lab6.coe file. Each location stores one 16-bit instruction. The instruction output was connected to mapper.sv. The mapper takes instruction[15:0] and separates it into rs1, rs2, rd, immediate, nzimm, offset, and opcode. These fields were sent to the instruction decoder from Lab 5. The decoder then controlled the register file, ALU source muxes, ALU operation, memory write signal, and write-back mux. The rest of the datapath still used the register file, ALU, data memory, and muxes from the previous labs.

For this lab, VIO was mainly used for observation instead of manual control. I used it to check PC, instruction, opcode, rd, rs1, rs2, immediate, nzimm, ALU output, register write data, and DataMemOut. This made it possible to verify that each BTN1 press moved to the next instruction and that the instruction was decoded correctly.

Results and Analysis

The hardware test showed that the PC and instruction memory worked correctly. After reset, PC returned to 00 and the first instruction was 05AD. Each BTN1 press advanced the PC by one and loaded the next instruction from the COE file. The instruction sequence observed in VIO was 05AD, C14C, 060D, C910, 8605, 40C8, 8D51, 4A80, and 8C69. This matched the expected instruction order from the prelab table.

The datapath results also matched the expected behavior. At PC = 00, instruction 05AD decoded as addi with opcode = 3, rd = 3, rs1 = 3, and nzimm = 0B. The ALU output was 000B, and this value was written back into the register file. At PC = 01, instruction C14C decoded as sw. The ALU calculated memory address 0004, and the value 000B from register 3 was written into data memory. Later, at PC = 05, instruction 40C8 loaded from the same memory address and VIO showed DataMemOut = 000B, proving that the earlier store worked.

The second store and load also worked. At PC = 03, instruction C910 stored 0003 into data

memory address 0010. At PC = 07, instruction 4A80 loaded from that address and VIO showed DataMemOut = 0003. The final original instruction, 8C69, performed an AND operation using 0003 and 000B. The ALU output was 0003, which matched the expected result.

I also added extra instructions to the COE file after the original program to test more instruction behavior. These included 0095, 0086, and 888C. They decoded as addi, slli, and andi. At PC = 09, instruction 0086 showed opcode = 9, rd = 1, rs1 = 1, and nzimm = 1. The ALU shifted the value in register 1 left by 1 and produced 000A. At PC = 10, instruction 888C showed opcode = 5 and immediate = 3. The ALU performed an immediate AND operation and produced 0002. These extra tests showed that the PC continued reading new instruction memory addresses correctly after the original nine instructions.

Conclusion & Recommendations

This lab showed how the computer can fetch and execute stored instructions automatically. The PC successfully addressed instruction memory, and the instruction memory output matched the COE file. The mapper separated each 16-bit instruction into the correct fields, and the decoder generated the correct datapath control signals. The VIO screenshots confirmed that the PC incremented properly, the correct instruction appeared at each address, and the datapath produced the expected register, ALU, and memory outputs.

Based on the results, it is recommended to test the design one instruction at a time using BTN1 and to save VIO screenshots for each PC value. The most useful signals to show are PC, instruction, opcode, rd, rs1, rs2, immediate or nzimm, ALU output, register write data, and DataMemOut. This makes it easier to prove both parts of the lab: instruction fetch and datapath execution.

Pre-Lab Section

The prelab required converting the COE machine code instructions into RISC-V style operations. The table below lists the original instructions and the additional test instructions added after the provided program.

Hex Instruction	Operation	Opcode	Rd	Rs1	Rs2	Immediate type and value	Explanation of what the instruction does
05AD	addi	4'b0011	3	3	x	nzimm = 11	Adds 11 to register address 3 and stores the result back in register address 3.
C14C	sw	4'b0001	x	2	3	immediate = 4	Stores the value from register address 3 into data memory address 4 using register address 2 as the base.
060D	addi	4'b0011	4	4	x	nzimm = 3	Adds 3 to register address 4 and stores the result back in register address 4.
C910	sw	4'b0001	x	2	4	immediate = 16	Stores the value from register address 4 into data memory address 16 using register address 2 as the base.
8605	srai	4'b1000	4	4	x	nzimm = 1	Arithmetic right shifts register address 4 by 1 and stores the result back in register address 4.
40C8	lw	4'b0000	2	1	x	immediate = 4	Loads the value from data memory address 4 into register address 2 using register address 1 as the base.
8D51	or	4'b0110	2	2	4	x	ORs register address 2 with register address 4 and stores the result in register address 2.
4A80	lw	4'b0000	0	5	x	immediate = 16	Loads the value from data memory address 16 into register address 0 using register address 5 as the base.
8C69	and	4'b0100	0	0	2	x	ANDs register address 0 with register address 2 and stores the result in register address 0.
0095	addi	4'b0011	1	1	x	nzimm = 5	Additional test instruction. Adds 5 to register address 1 and stores the result back in register address 1.
0086	slli	4'b1001	1	1	x	nzimm = 1	Additional test instruction. Shifts register address 1 left by 1 and stores the result back in register address 1.
888C	andi	4'b0101	1	1	x	immediate = 3	Additional test instruction. ANDs register address 1 with immediate value 3 and stores the result back in register address 1.

Table 1: Lab 6 instruction conversion table

Code and Testing

The main new code written for this lab was the Program Counter. It resets to 0 when BTN0 is pressed and increments by 1 on every BTN1 clock press.

```
`timescale 1ns / 1ps
module program_counter(
    input logic clk,
    input logic rst,
    output logic [7:0] pc
);

always_ff @(posedge clk or posedge rst) begin
    if (rst)
        pc <= 8'b00000000;
    else
        pc <= pc + 1;
end
endmodule
```

Testing was done by resetting the PC to 0 and pressing BTN1 one time per instruction. The expected PC and instruction sequence is shown below.

Step	Reset	Clock action	Expected PC	Expected instruction	Expected operation
0	1	none	0	reset/cleared	reset state
1	0	after reset release	0	05AD	addi
2	0	BTN1 press	1	C14C	sw
3	0	BTN1 press	2	060D	addi
4	0	BTN1 press	3	C910	sw
5	0	BTN1 press	4	8605	srai
6	0	BTN1 press	5	40C8	lw
7	0	BTN1 press	6	8D51	or
8	0	BTN1 press	7	4A80	lw
9	0	BTN1 press	8	8C69	and
10	0	BTN1 press	9	0095	addi
11	0	BTN1 press	A	0086	slli
12	0	BTN1 press	B	888C	andi

Table 2: Expected PC and instruction output

Appendix A: Design Diagram

The diagram below shows the Lab 6 computer structure. The main new flow is PC Logic to Instruction Memory to Mapper to Instruction Decoder.

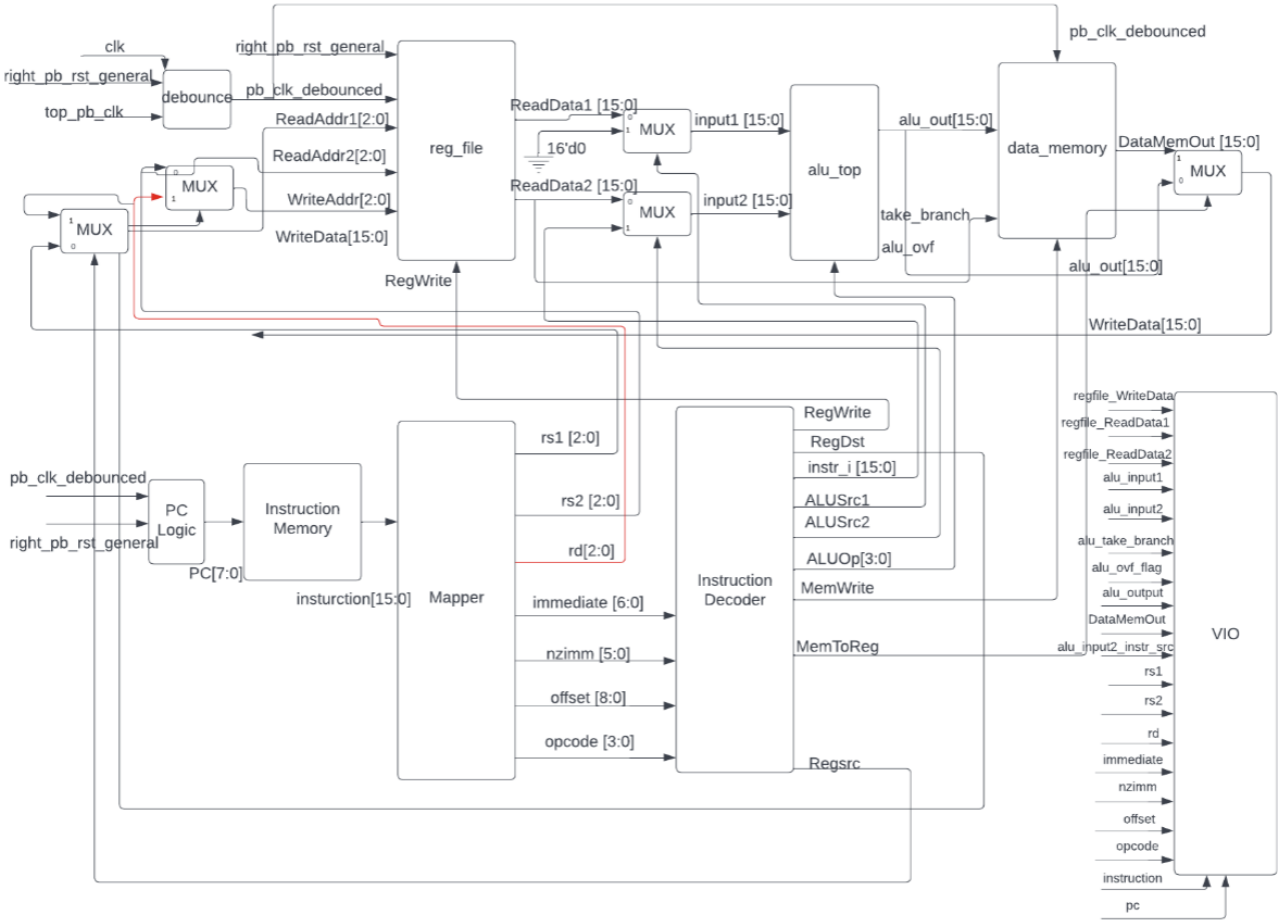


Figure A1: Computer with Instruction Memory and Program Counter

Appendix B: Program Files and Code

Main design and support files used in this lab: alu_instrmem_top.sv, program_counter.sv, mapper.sv, instruction_decoder.sv, regfile.sv, alu.sv, TwoToOneMux.sv, debounce.sv, adaptor_display.sv, instruction_memory IP, data_mem IP, vio_0 IP, testing_lab6.coe, and lab_6_constraint.xdc.

B.1 Program Counter Code

```
`timescale 1ns / 1ps

module program_counter(
    input logic clk,
    input logic rst,
    output logic [7:0] pc
);

always_ff @(posedge clk or posedge rst) begin
    if (rst)
        pc <= 8'b00000000;
    else
        pc <= pc + 1;
end

endmodule
```

B.2 COE File Contents

```
memory_initialization_radix=16;
memory_initialization_vector=05ad,c14c,060d,c910,8605,40c8,8d51,4a80,8c69,0095,0086,888c;
```

Appendix C: Hardware VIO Output Screen Captures

The following VIO screenshots show the PC value, instruction, opcode, register fields, immediate fields, ALU output, register write data, and data memory output during hardware testing.

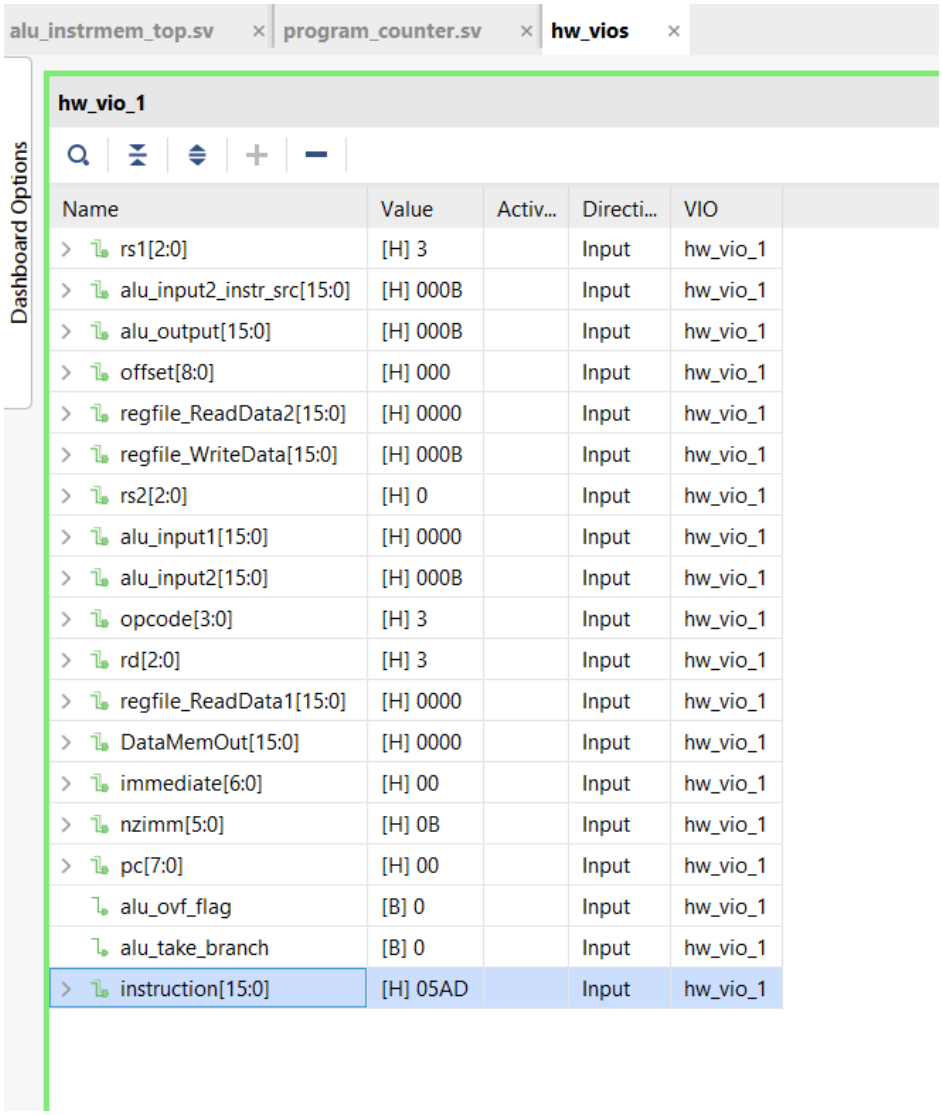


Figure C1: PC = 00, instruction = 05AD, addi instruction decoded correctly

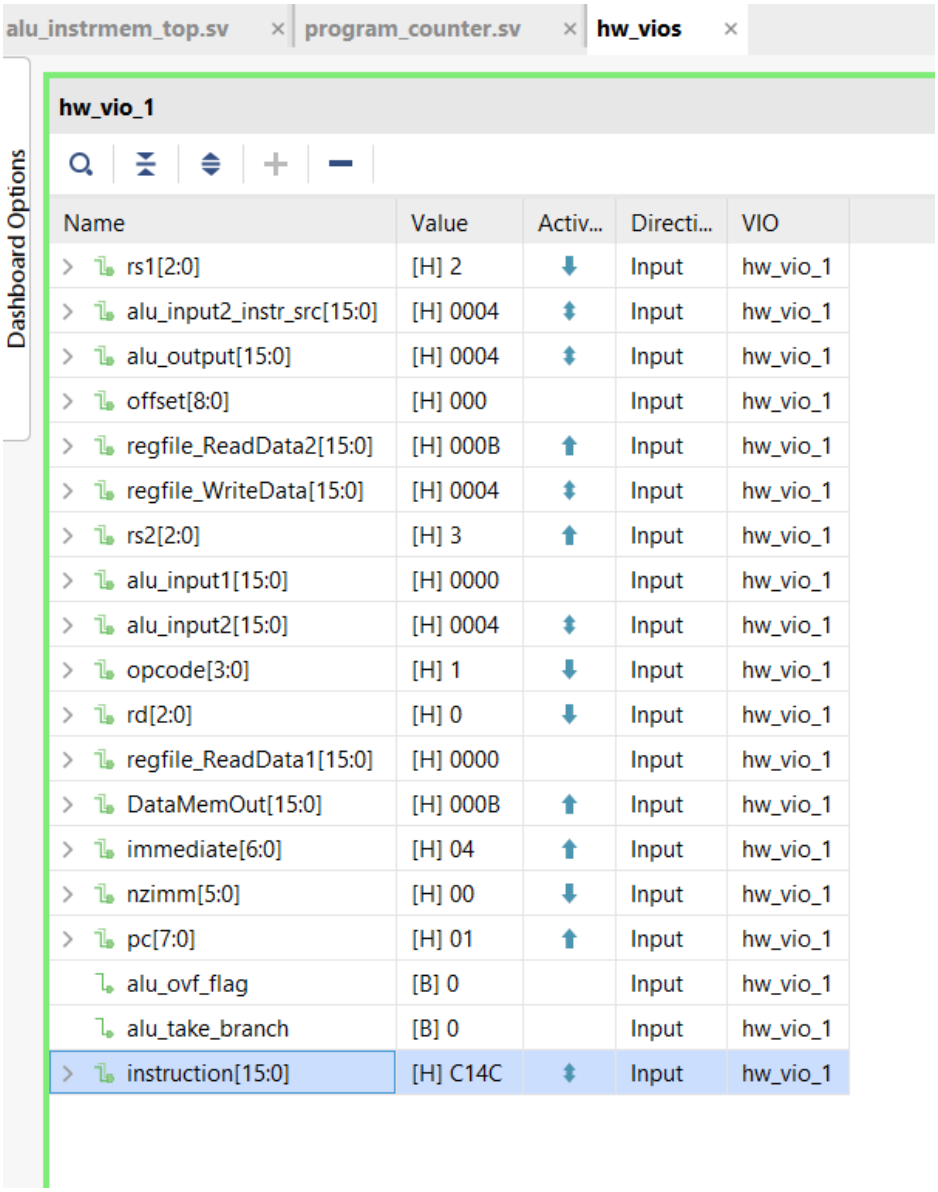


Figure C2: PC = 01, instruction = C14C, store word instruction decoded correctly

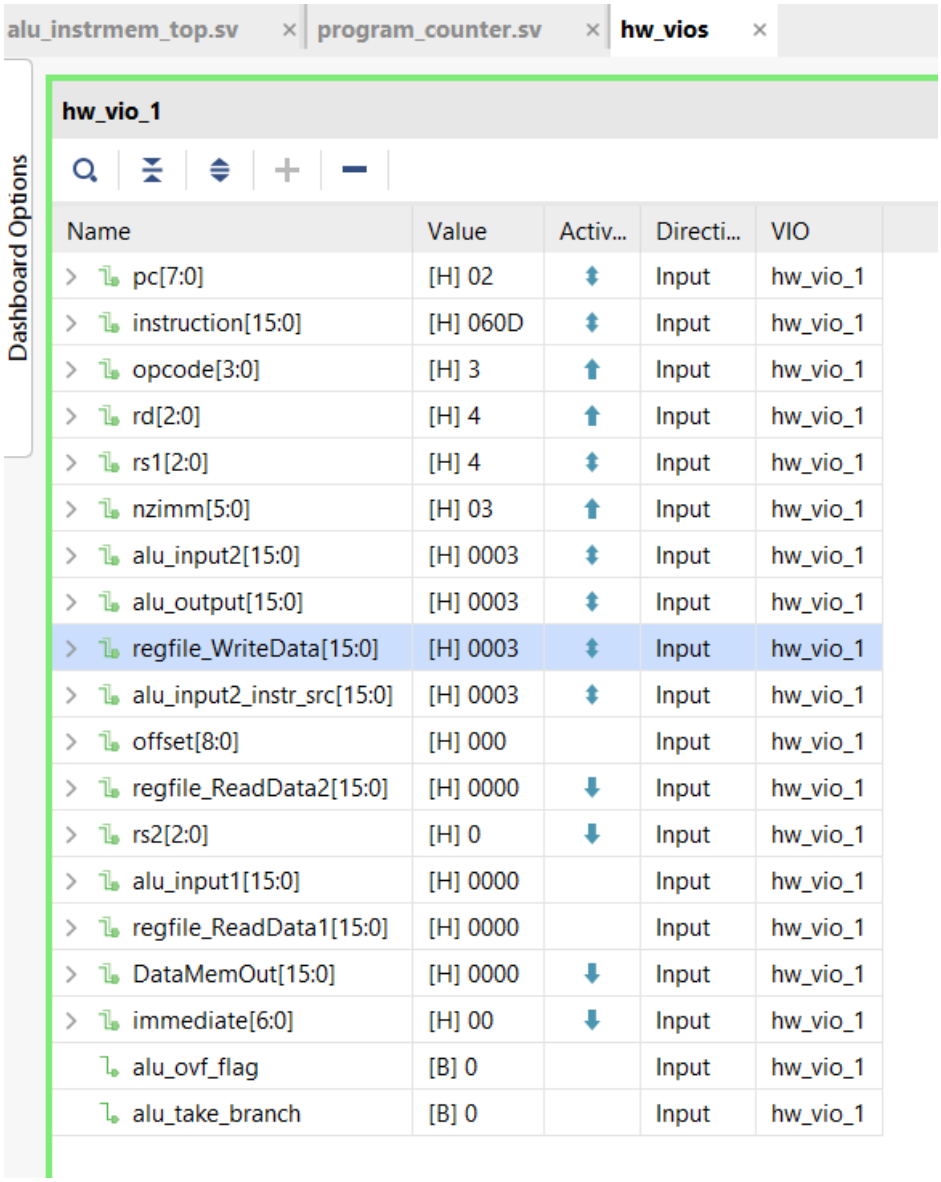


Figure C3: PC = 02, instruction = 060D, addi instruction decoded correctly

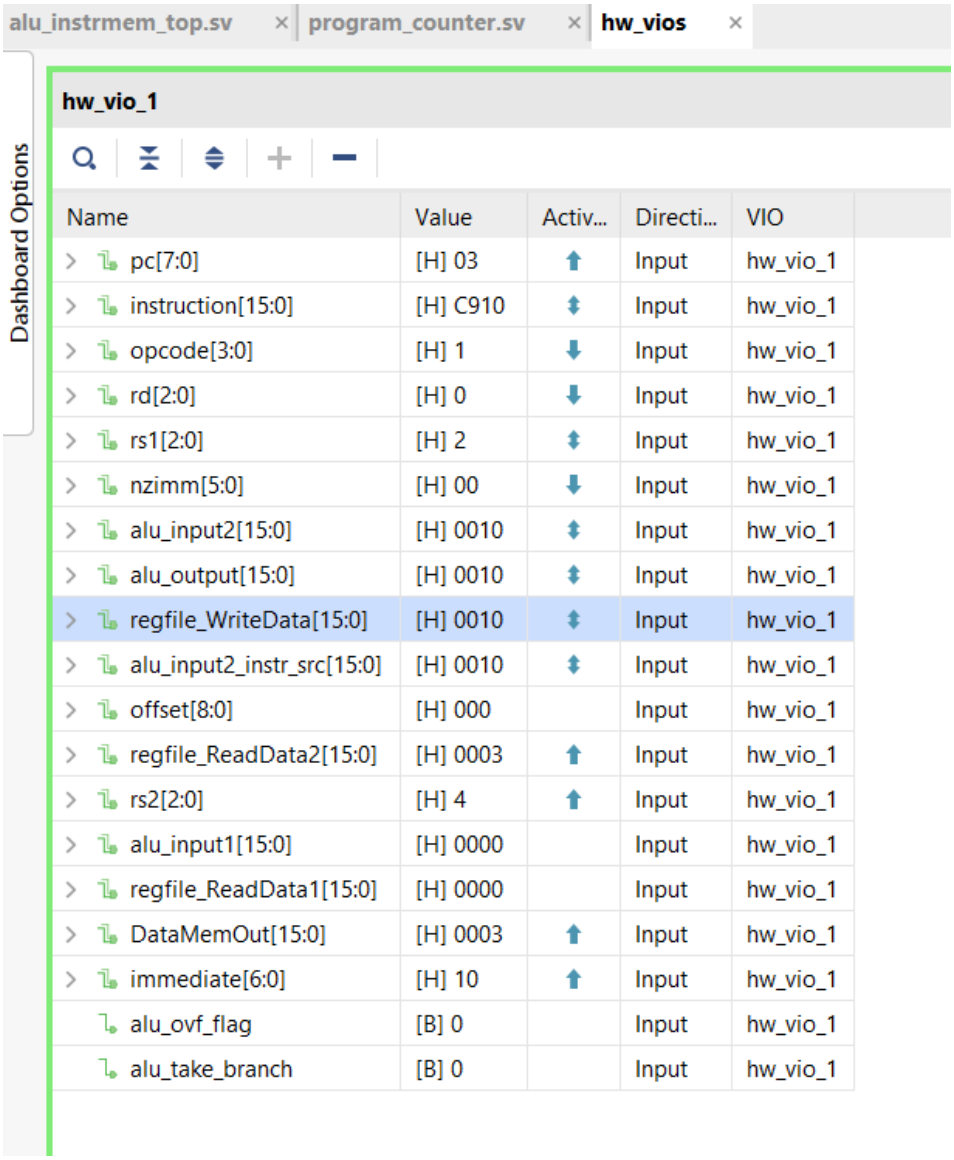


Figure C4: PC = 03, instruction = C910, store word instruction decoded correctly

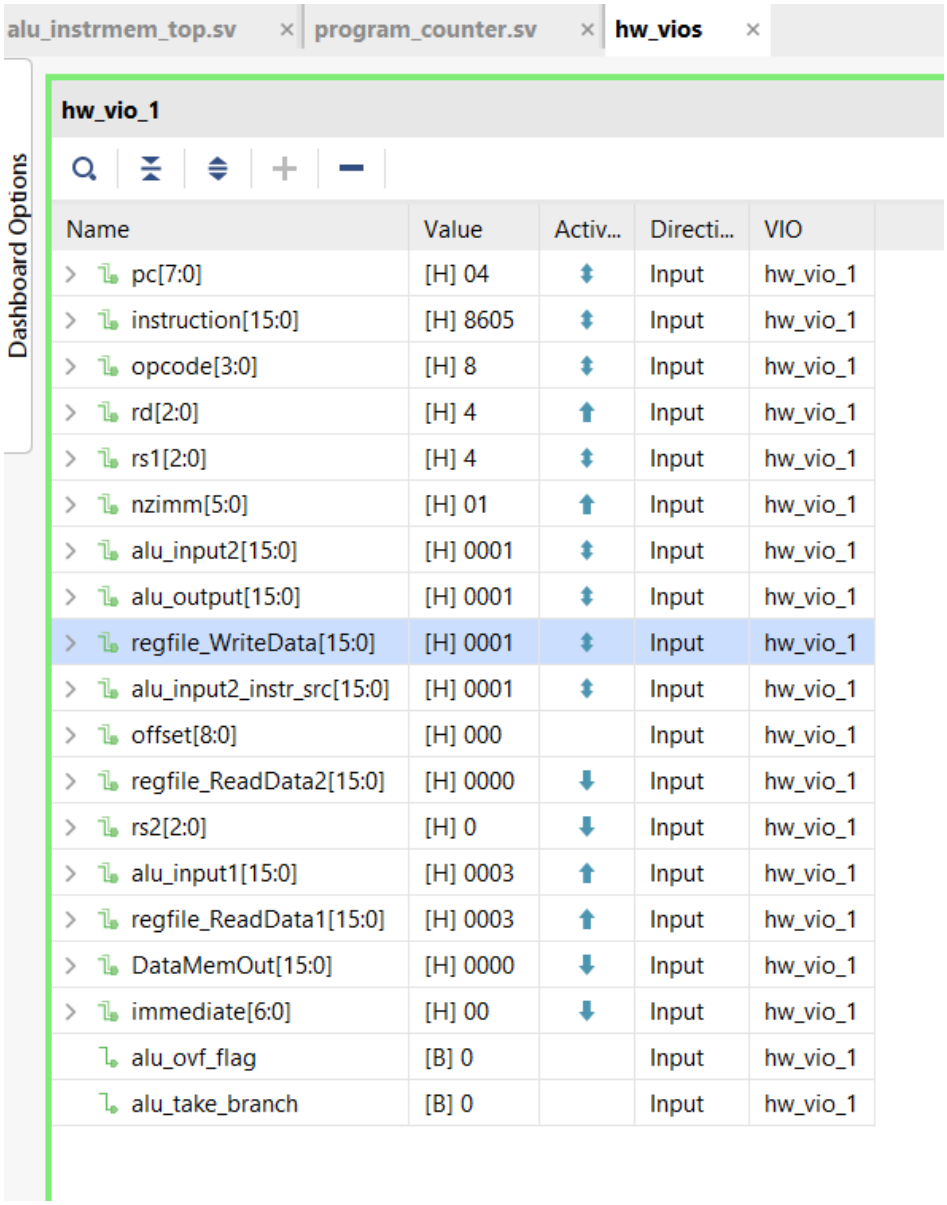


Figure C5: PC = 04, instruction = 8605, srai instruction decoded correctly

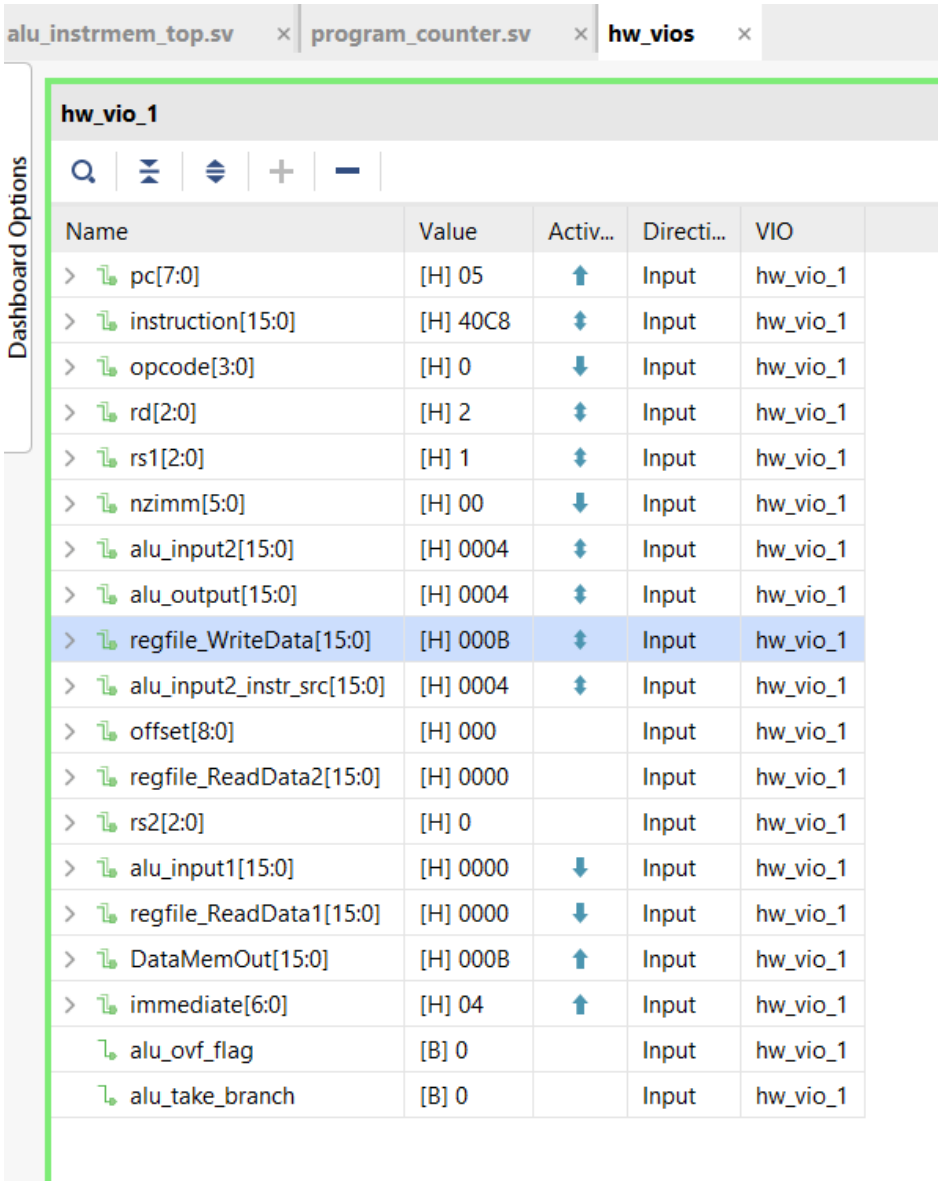


Figure C6: PC = 05, instruction = 40C8, lw instruction loaded DataMemOut = 000B

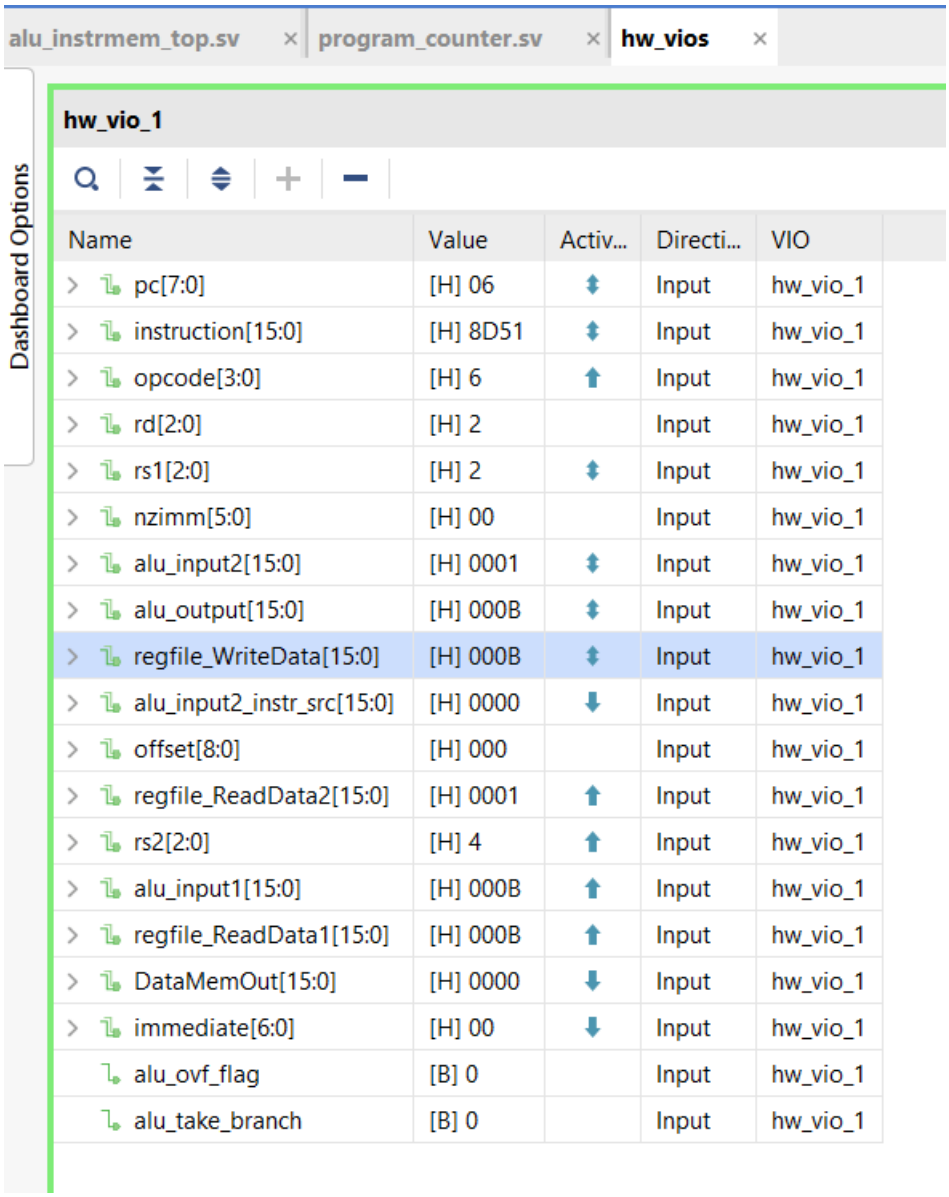


Figure C7: PC = 06, instruction = 8D51, or instruction decoded and executed correctly

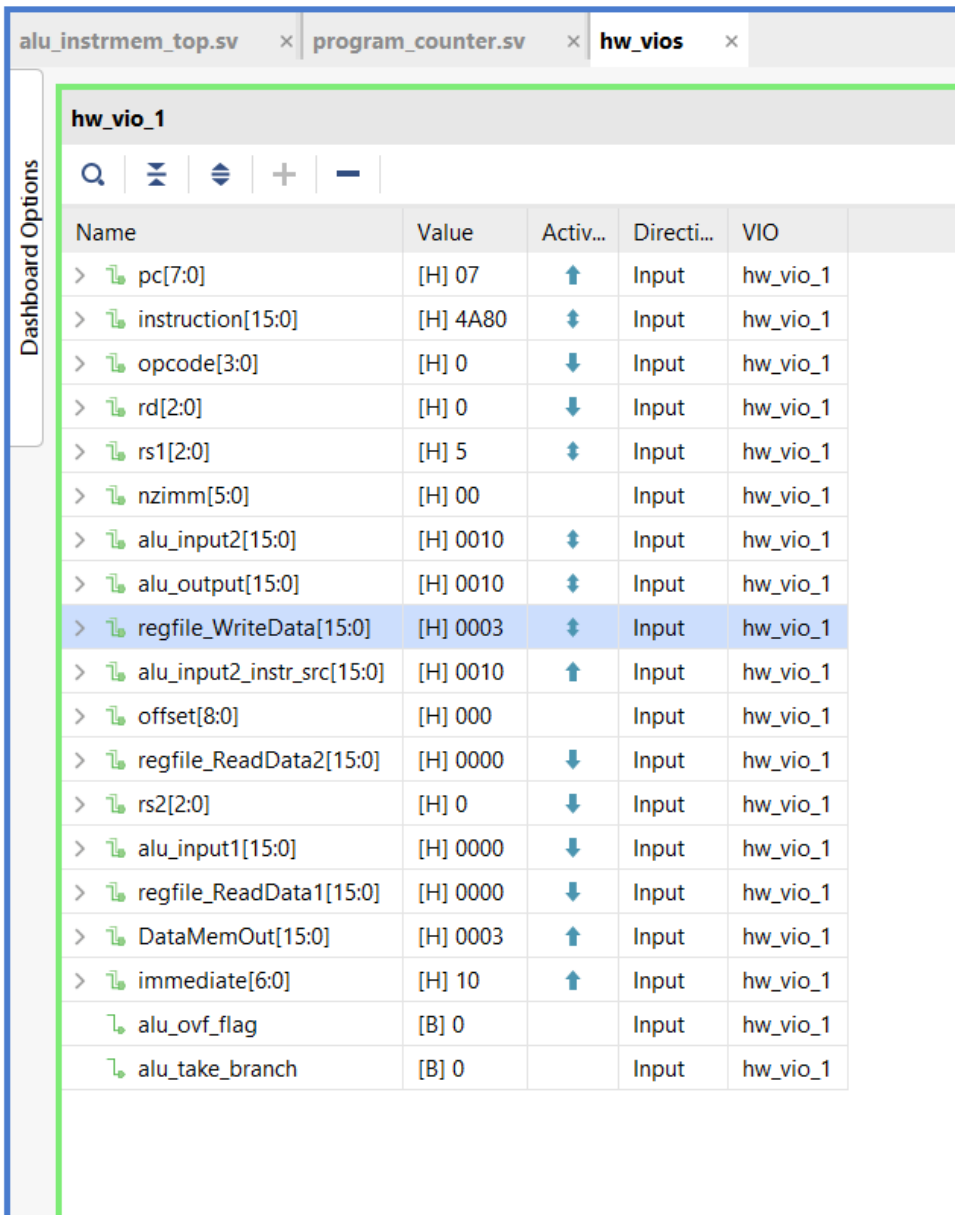


Figure C8: PC = 07, instruction = 4A80, lw instruction loaded DataMemOut = 0003

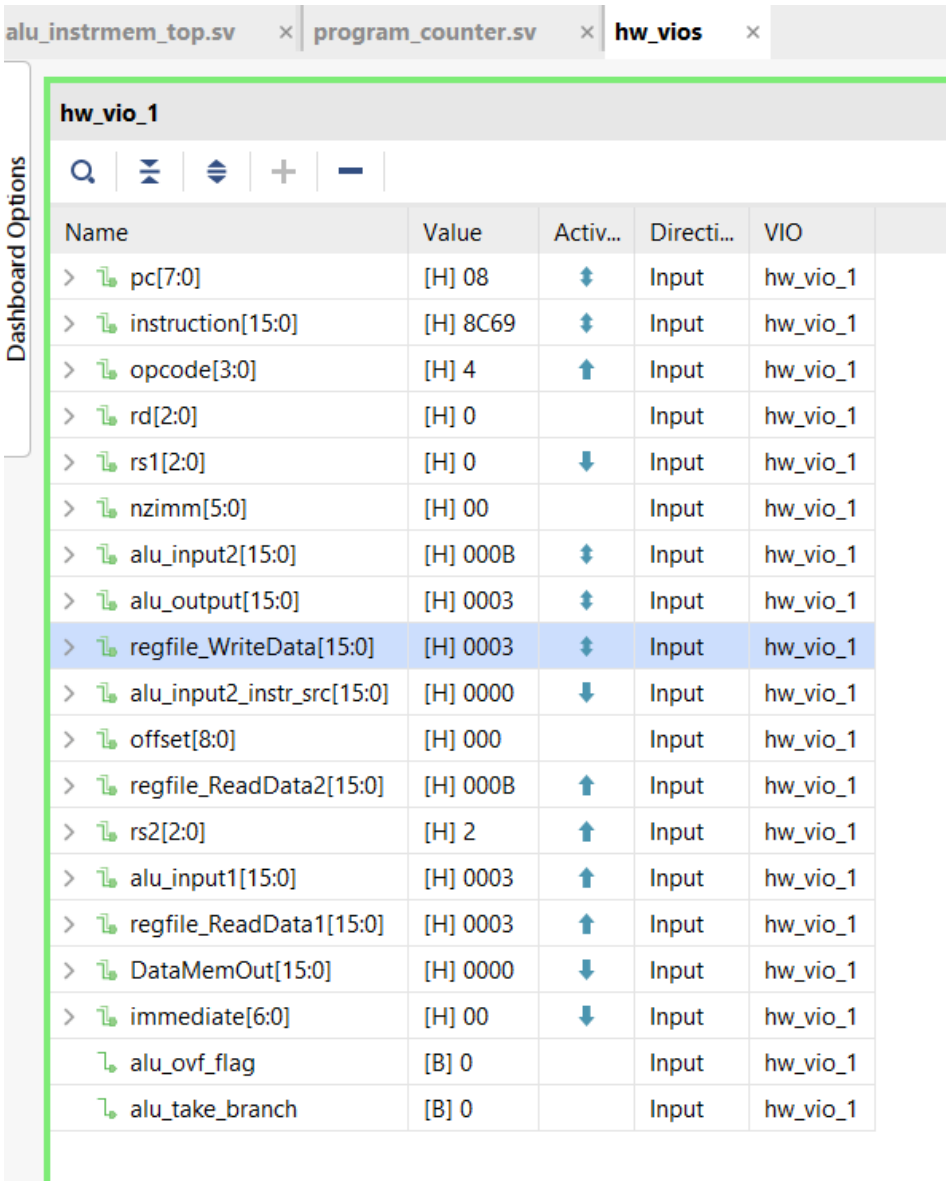


Figure C9: PC = 08, instruction = 8C69, and instruction decoded and executed correctly

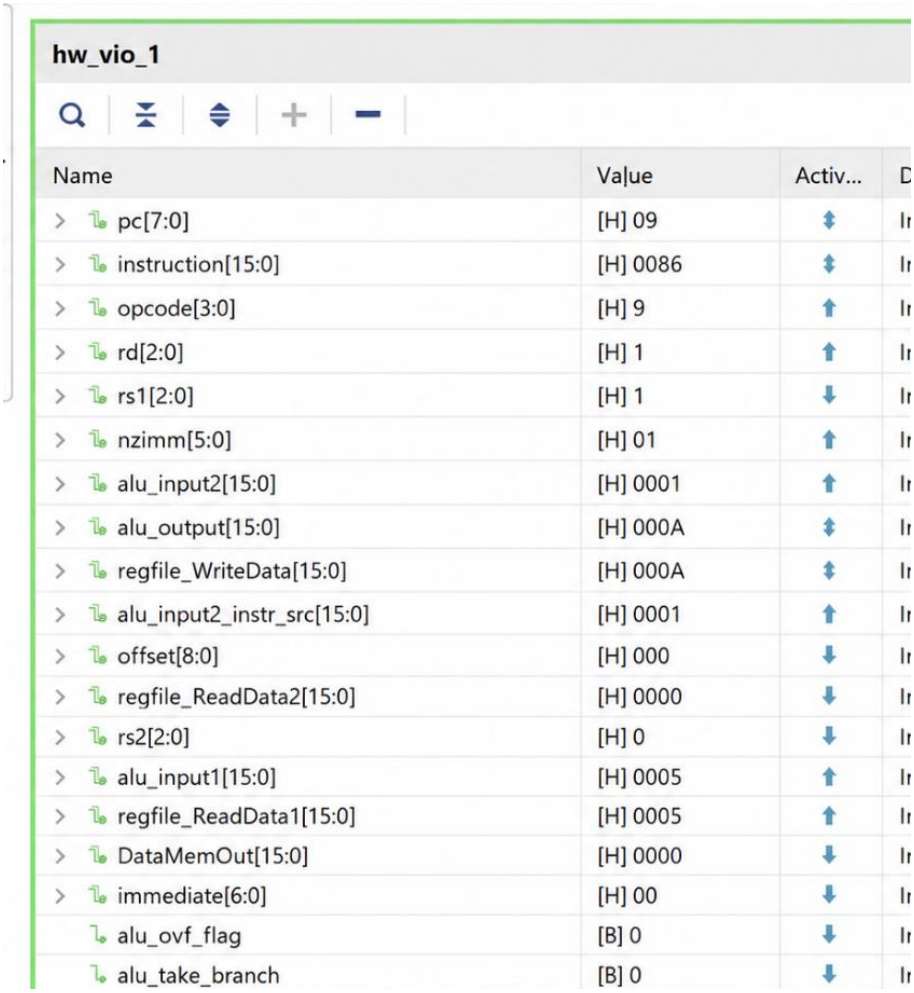


Figure C10: PC = 9, instruction = 0086, slli instruction decoded and executed correctly

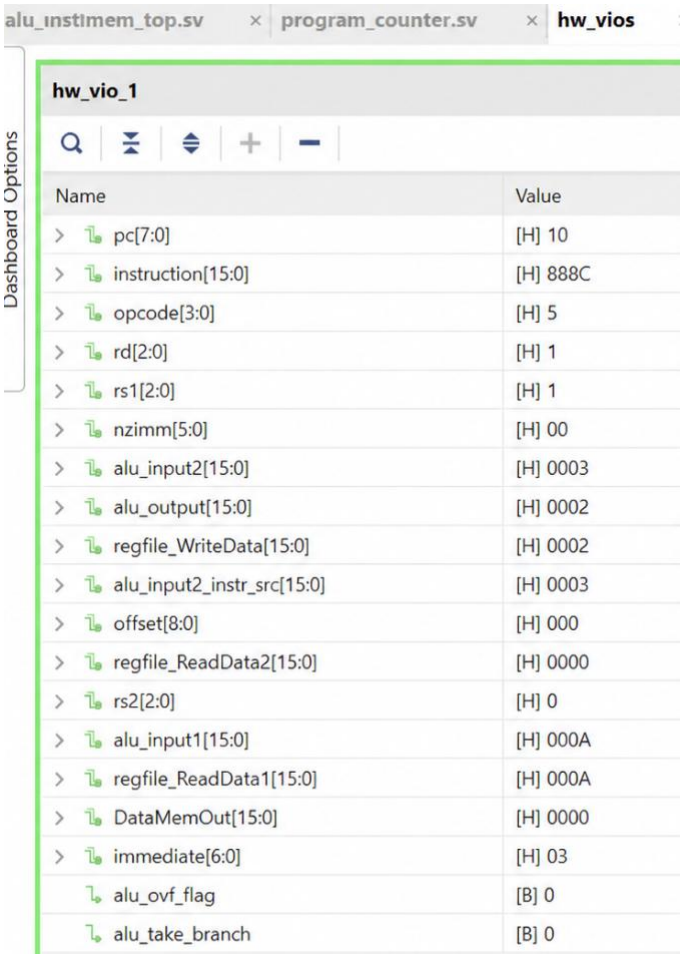


Figure C11: PC = 10, instruction = 888C, andi instruction decoded and executed correctly.